

TP5 – Sistema 1: Implementación del motor de simulación

Kuramoto – Java

Simulación de Sistemas – 2026Q1

Motor de simulación del modelo de Kuramoto en Java, archivo único `motor/KuramotoSim.java` ($\sim 6,4$ kB, bajo el límite de 20 kB del enunciado). Sólo motor: el análisis y la animación se hacen aparte sobre los CSV generados.

1. Estructura del código

Una sola clase `KuramotoSim` con un `main` y métodos auxiliares estáticos. Sin estado global ni dependencias externas (solo `java.io` y `java.util`).

Método	Responsabilidad
<code>main(args)</code>	Parsea CLI, inicializa ω_i , $\theta_i(0)$ y A_{ij} , abre el CSV, corre el loop principal.
<code>parseArgs(args)</code>	Lee flags <code>-clave valor</code> a un Map.
<code>i(...)</code> / <code>d(...)</code>	Helpers para enteros y dobles con default.
<code>buildNeighbors(...)</code>	Construye la lista de vecinos por nodo (<code>int[] []</code>) según topología.
<code>deriv(...)</code>	Evalúa $f(\theta)_i = \omega_i + K \sum_{j \in \text{vec}(i)} \sin(\theta_j - \theta_i)$.
<code>writeRow(...)</code>	Calcula $r(t)$ y escribe una fila del CSV.

Estructura de datos: listas de vecinos

En vez de guardar A_{ij} como matriz densa $N \times N$ (250 000 entradas para $N = 500$), guardamos para cada nodo i un **arreglo de índices de sus vecinos**. Esto: (i) recorre sólo los vecinos reales en `deriv` (rápido para redes esparsas), (ii) es equivalente para el caso completo, (iii) aprovecha que el grafo no cambia durante la simulación.

Reproducibilidad: dos semillas

- `-seed`: controla ω_i y $\theta_i(0)$. Misma semilla $\Rightarrow$ mismas condiciones iniciales *independientemente de la topología*.
- `-netSeed`: controla el sorteo de A_{ij} (sólo `random`). Default = `seed`.

Facilita comparar topologías sobre la misma condición inicial y promediar separadamente sobre realizaciones de red y de condiciones iniciales.

2. Parámetros (CLI)

Todos parametrizables vía `-clave valor`.

Parámetros del modelo

Flag	Tipo	Default	Descripción
-N	int	500	Número de osciladores (consigna: $N > 500$).
-K	double	1.0	Acoplamiento global $K \in [0, 1]$.
-topology	string	complete	complete/random/ring.
-p	double	0.5	Probabilidad de conexión (sólo random).
-v	int	1	Vecindad del anillo (sólo ring), $v \in [1, 10]$.
-muOmega	double	1.0	Media de la normal para ω_i .
-sigmaOmega	double	0.1	Desvío estándar de la normal para ω_i .

Parámetros numéricos y de salida

Flag	Tipo	Default	Descripción
-dt	double	0.01	Paso de integración (fijo). Probar 10^{-x} , $x \in \{1, \dots, 4\}$.
-tSim	double	100.0	Tiempo total simulado.
-seed	long	42	Semilla para ω_i y $\theta_i(0)$.
-netSeed	long	= seed	Semilla para A_{ij} (sólo random).
-output	string	kuramoto.csv	Archivo de salida.
-dumpEvery	int	1	Vuelca cada n pasos al CSV.
-dumpPhases	bool	true	Si false, sólo t,r (mucho más liviano).

3. Formato del archivo de salida (CSV)

```
# N=500 K=1.000000 dt=0.010000 tSim=100.000000 topology=COMPLETE p=0.500000 v=1 muOmega=1.000000
  sigmaOmega=0.100000 seed=42 netSeed=42 dumpEvery=1
t,r,theta_0,theta_1,...,theta_{N-1}
0.0,0.0763...,1.4234...,3.1415...,...
0.01,0.0791...,1.4334...,3.1525...,...
...
```

- Primera línea: comentario con todos los parámetros (autodocumenta el archivo).
- Segunda línea: nombres de columna.
- Filas siguientes: valores. Si `-dumpPhases false`, sólo `t` y `r`.

$r(t)$ se computa dentro del motor en `writeRow` ($O(N)$ por fila): el postproceso lo lee directo, y `-dumpPhases false` habilita un modo “solo r ” liviano para los barridos masivos.

4. Compilación y ejecución

```
cd motor
javac KuramotoSim.java

# Red completa, K=0.5
java KuramotoSim --N 500 --K 0.5 --topology complete \
  --dt 0.01 --tSim 100 \
  --seed 42 --output complete_K0.5_seed42.csv \
  --dumpEvery 10 --dumpPhases true

# Red aleatoria, K=1, p=0.3, semillas separadas
java KuramotoSim --N 500 --K 1.0 --topology random --p 0.3 \
```

```

--dt 0.01 --tSim 100 \
--seed 42 --netSeed 7 \
--output random_p0.3_K1_s42_n7.csv \
--dumpPhases false

# Red anillo, v=3
java KuramotoSim --N 500 --K 0.7 --topology ring --v 3 \
--dt 0.01 --tSim 200 \
--seed 42 --output ring_v3_K0.7.csv

```

5. Algoritmo

Inicialización:

1. $\omega_i \sim \mathcal{N}(1, 0, 1^2)$ con la RNG de `seed` (vía `nextGaussian()`).
2. $\theta_i(0) \sim \mathcal{U}[0, 2\pi)$ con la misma RNG.
3. Lista de vecinos según topología (con RNG de `netSeed` si corresponde).

Loop principal – RK4: con $f(\theta)_i = \omega_i + K \sum_{j \in \text{vec}(i)} \sin(\theta_j - \theta_i)$,

$$\mathbf{k}_1 = f(\boldsymbol{\theta}), \quad \mathbf{k}_2 = f(\boldsymbol{\theta} + \frac{dt}{2}\mathbf{k}_1), \quad \mathbf{k}_3 = f(\boldsymbol{\theta} + \frac{dt}{2}\mathbf{k}_2), \quad \mathbf{k}_4 = f(\boldsymbol{\theta} + dt\mathbf{k}_3)$$

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \frac{dt}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4)$$

El sistema es autónomo; `deriv` no recibe argumento de tiempo. Los buffers `k1..k4` y `tmp` se alocan una sola vez fuera del loop.

Cálculo de r :

$$c_x = \frac{1}{N} \sum_j \cos \theta_j, \quad c_y = \frac{1}{N} \sum_j \sin \theta_j, \quad r = \sqrt{c_x^2 + c_y^2}$$

Construcción de vecinos:

- *complete*: `nbr[i] = {0..N-1} \ {i}`.
- *random*: para cada par (i, j) con $i < j$, sortear `nextDouble() < p`; si sí, agregar j a `nbr[i]` y i a `nbr[j]` (red **no dirigida** por convención: el enunciado no lo exige explícitamente).
- *ring*: `nbr[i] = {(i-v) mod N, ..., (i+v) mod N} \ {i}` (exactamente $2v$ vecinos). Aritmética modular con `((i-d) % N + N) % N` para manejar negativos en Java.

6. Lo que el motor *no* hace (a propósito)

- No grafica ni anima.
- No calcula τ_s ; emite $r(t)$, el umbral y la detección del estacionario van en postproceso.
- No promedia sobre realizaciones (ejecutar el motor varias veces con distintas `-seed`).
- No barre K, p, v automáticamente (cada combinación se corre como proceso separado, fácil de paralelizar con `xargs -P` o un script).

Esto sigue el requisito del enunciado: “*el análisis y módulo de animación se ejecuta en forma independiente tomando estos archivos de texto como input.*”